

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN

-----&-----



ĐỒ ÁN MÔN HỌC
CÔNG NGHỆ PHẦN MỀM
XÂY DỰNG HỆ THỐNG
VẬN HÀNH VÀ QUẢN LÝ SÂN THỂ THAO

GVHD: TS. Nguyễn Bảo Ân

Nhóm sinh viên thực hiện:

Nguyễn Gia Nguyên - 110123239 - DA23TTB

Nguyễn Thị Hân Du - 110123079 - DA23TTB

Nguyễn Hữu Lộc - 110123124 - DA23TTB

Vĩnh Long, tháng 8 năm 2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, Ngày tháng 08 năm 2026

Giáo viên hướng dẫn

LỜI CẢM ƠN

Chúng em xin chân thành gửi lời cảm ơn đến giảng viên hướng dẫn - TS. Nguyễn Bảo Ân đã nhiệt tình giảng dạy trong quá trình học tập môn Công nghệ phần mềm và tận tâm chỉ dẫn cho đồ án môn học. Chúng em vô cùng cảm ơn những kiến thức bổ ích của thầy về bộ môn này. Nhờ đó chúng em đã có thêm nền tảng vững chắc cho môn học và góp phần hoàn thành suôn sẻ đồ án kết thúc môn.

Chúng em cũng xin gửi lời cảm ơn đến Đại học Trà Vinh nói chung, Trường Kỹ thuật và Công nghệ nói riêng đã tạo điều kiện cho chúng em được học tập và rèn luyện kỹ năng cần thiết.

Do kiến thức và kỹ năng còn hạn chế, đồ án của chúng em sẽ vẫn còn thiếu sót, nhưng chúng em đã cố gắng hoàn thành trong khả năng của mình, mong thầy góp ý để chúng em hoàn thiện và phát triển tốt hơn.

Chúng em xin chân thành cảm ơn!

Vĩnh Long, ngày ... tháng 8 năm 2026

Nhóm sinh viên thực hiện

TÓM TẮT ĐỀ TÀI

Bạn đã bao giờ đặt sân bóng qua tin nhắn, đến nơi mới ngỡ ngàng vì sân đã có người khác đặt trước do chủ sân ghi nhầm vào sổ tay? Đó là câu chuyện quen thuộc mà hàng nghìn cơ sở sân thể thao vừa và nhỏ tại Việt Nam vẫn gặp phải mỗi ngày - nơi việc quản lý lịch đặt sân, thu tiền và theo dõi doanh thu vẫn dựa vào sổ sách, tin nhắn và cả... trí nhớ của nhân viên.

Đề tài “Xây dựng hệ thống vận hành và quản lý sân thể thao” ra đời để giải quyết triệt để bài toán đó. Không đơn thuần là một website đặt sân, hệ thống được thiết kế như một “bộ não vận hành” thông minh: tự động phát hiện và ngăn chặn trùng lịch chỉ trong tích tắc nhờ thuật toán kiểm tra chồng khung giờ trên MongoDB Aggregation; tự động giải phóng những lượt đặt sân bị “bỏ quên” chưa thanh toán để nhường chỗ cho khách hàng khác; và trực quan hóa toàn bộ khung giờ trống/đã đặt qua một giao diện lưới sinh động - khách hàng chỉ cần vài cú chạm là hoàn tất việc đặt sân.

Phía sau trải nghiệm mượt mà đó là một kiến trúc RESTful API bài bản theo mô hình phân lớp, được bảo vệ bởi cơ chế xác thực JWT và phân quyền RBAC chặt chẽ theo ba vai trò Khách hàng – Nhân viên – Quản trị viên. Toàn bộ hệ thống - từ cơ sở dữ liệu, backend đến giao diện - được đóng gói gọn gàng bằng Docker, sẵn sàng triển khai chỉ với một dòng lệnh duy nhất.

Từ một vấn đề rất đời thường, đề tài đã được hiện thực hóa thành một sản phẩm phần mềm hoàn chỉnh và thực chiến, minh chứng cho khả năng vận dụng trọn vẹn quy trình phân tích – thiết kế – xây dựng – kiểm thử một hệ thống web hiện đại.

MỤC LỤC

LỜI CẢM ƠN.....	i
TÓM TẮT ĐỀ TÀI.....	ii
MỤC LỤC.....	iii
CHƯƠNG 1. GIỚI THIỆU.....	1
1.1. Đặc tả ứng dụng.....	1
1.2. Mục tiêu của ứng dụng.....	1
1.3. Phạm vi ứng dụng.....	2
1.3.1. Phạm vi trong đề tài.....	2
1.3.2. Phạm vi ngoài đề tài.....	3
1.4. Đối tượng sử dụng.....	3
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	4
2.1. Agile.....	4
2.2. Scrum.....	4
2.3. Công nghệ sử dụng.....	5
2.4. Kiến trúc ứng dụng.....	6
CHƯƠNG 3. XÁC ĐỊNH NHU CẦU.....	7
3.1. Đặc tả mục tiêu dự án.....	7
3.2. Xác định Personas.....	7
3.3. Xây dựng User Stories.....	8
3.4. Xác định Product Backlog.....	9
3.5. Xác định nhu cầu phi chức năng.....	10
3.6. Các tác nhân (Actor).....	11
3.7. Đặc tả Use Case chi tiết.....	11
UC-01: Đặt sân (Create Booking).....	11
UC-02: Thanh toán booking (Create Payment).....	12
UC-03: Xác nhận / cập nhật booking.....	13
UC-04: Hủy booking (Cancel Booking).....	13
UC-05: Tự động hết hạn booking chờ thanh toán.....	14
UC-06: Thông kê báo cáo doanh thu.....	14
UC-07: Tra cứu khung giờ đã đặt (Availability).....	15
CHƯƠNG 4. LẬP KẾ HOẠCH HIỆN THỰC HÓA BẰNG SCRUM.....	16
4.1. Nguyên tắc sắp xếp ưu tiên Product Backlog.....	16
4.2. Phân chia Sprint.....	16

4.3. Sprint Backlog và phân công.....	17
4.4. Ước lượng Story Points	18
4.5. Lập kế hoạch và quản lý Sprint trên Jira	19
CHƯƠNG 5. HIỆN THỰC HÓA KẾ HOẠCH.....	21
5.1. Thiết kế kiến trúc ứng dụng & Thư mục mã nguồn	21
5.2. Thiết kế Database.....	22
a) Model User (Lưu trữ người dùng và vai trò).....	22
b) Model FieldType (Loại sân thể thao).....	22
c) Model Field (Sân thể thao cụ thể).....	22
d) Model Booking (Đơn đặt sân).....	23
e) Model Payment (Thông tin thanh toán).....	23
f) Sơ đồ thực thể quan hệ ERD	23
5.3. Thiết thiết REST API & Phân quyền JWT.....	24
5.4. Hiện thực các Sprint.....	25
5.4.1. Sprint 1 – Authentication & User Management	25
5.4.2. Sprint 2 – Core Database, Field Management, Booking & Payment.....	26
5.4.3. Sprint 3 – React UI, Dashboard & Docker.....	28
5.5. Xây dựng giao diện bằng mã nguồn HTML/CSS, quản lý mã nguồn Git/GitHub.....	30
5.6. Kiểm thử hệ thống (Test Cases)	32
5.7. Nhận xét kết quả kiểm thử.....	36
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	37
6.1. Kết quả đạt được.....	37
6.2. Hạn chế.....	37
6.3. Hướng phát triển.....	38
6.4. Kết luận chung.....	39
TÀI LIỆU THAM KHẢO.....	40

DANH MỤC HÌNH ẢNH

Hình 3.1	Sơ đồ chuyển trạng thái của Booking	12
Hình 3.2	Sơ đồ tuần tự cho luồng thanh toán	13
Hình 4.1	Scrum Board trên Jira	19
Hình 4.2	Sơ đồ ERD	23
Hình 5.1	Các endpoint API đã được tải liệu hóa thành công	25
Hình 5.2	Giao diện lưới khung giờ đặt sân	27
Hình 5.3	Chạy hệ thống trên Docker	29
Hình 5.4	Giao diện thanh toán	29
Hình 5.5	Giao diện quản lý loại sân của Admin	30
Hình 5.6	Giao diện quản lý thanh toán của Admin	30
Hình 5.7	Giao diện báo cáo thống kê	31
Hình 5.8	Giao diện quản lý người dùng và tạo Staff	31

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặc tả ứng dụng

Đề tài được xây dựng nhằm hiện thực hóa một hệ thống quản lý và vận hành sân thể thao theo phương pháp Agile – Scrum. Hệ thống hỗ trợ các nghiệp vụ chính gồm quản lý người dùng và xác thực, quản lý loại sân và sân, đặt sân, kiểm tra trùng lịch, thanh toán, quản lý booking và cung cấp giao diện quản trị.

Dự án được tổ chức theo mô hình nhóm 03 thành viên. Công việc được chia theo ba nhóm chính: Thành viên 1 phụ trách Authentication & User Management; Thành viên 2 phụ trách Core Booking Engine & Database; Thành viên 3 phụ trách React UI & Dashboard. Sự phối hợp của các thành viên được thực hiện thông qua công cụ quản lý dự án trực quan Jira, và mã nguồn dự án được đồng bộ thông qua Git/GitHub, giúp nâng cao tinh thần làm việc nhóm và tiếp cận môi trường làm việc thực tế trong các doanh nghiệp phát triển phần mềm.

1.2. Mục tiêu của ứng dụng

Đề tài hướng đến xây dựng một hệ thống web hoàn chỉnh, gồm phần backend cung cấp REST API và phần frontend phục vụ ba nhóm người dùng, nhằm số hóa toàn bộ quy trình vận hành của một cơ sở cho thuê sân thể thao. Cụ thể, hệ thống cần đạt được các mục tiêu sau:

- Xây dựng một hệ thống quản lý sân thể thao có cấu trúc rõ ràng và có khả năng vận hành các nghiệp vụ cốt lõi.
- Áp dụng quy trình Agile – Scrum vào quá trình phân tích, lập kế hoạch, phát triển và theo dõi tiến độ.
- Xây dựng Backend cung cấp API phục vụ xác thực, quản lý sân, booking và thanh toán.
- Xây dựng Frontend React cung cấp giao diện cho khách hàng và khu vực quản trị.
- Áp dụng JWT và middleware để xác thực người dùng và kiểm soát quyền truy cập.
- Cho phép khách hàng tự tra cứu thông tin sân, đặt sân theo khung giờ trống và thanh toán trực tuyến (chuyển khoản có ảnh xác nhận) hoặc tiền mặt tại quầy.

- Tự động kiểm tra và ngăn chặn tình trạng trùng lịch đặt sân (hai booking cùng sân, cùng ngày, có khung giờ giao nhau) thông qua các giải thuật so sánh thời gian tối ưu.
- Tự động giải phóng khung giờ nếu khách hàng đặt sân nhưng không thanh toán trước hạn giữ chỗ quy định (đến 1 giờ sau khi khung giờ đã đặt kết thúc).
- Hỗ trợ nhân viên (staff) quản lý sân, loại sân, xác nhận booking và duyệt thanh toán.
- Hỗ trợ quản trị viên (admin) quản lý toàn bộ tài khoản người dùng, tài khoản nhân viên, và trích xuất báo cáo thống kê doanh thu theo khoảng thời gian.
- Cung cấp tài liệu API tự động thông qua Swagger để thuận tiện cho việc kiểm thử và tích hợp.
- Đóng gói hệ thống bằng Docker và Docker Compose theo kế hoạch Sprint, sẵn sàng triển khai trên các môi trường đám mây VPS.

1.3. Phạm vi ứng dụng

Phạm vi chức năng được xác định từ các Product Backlog và Sprint Task được cung cấp, bao gồm Authentication & User Management, Field/FieldType, Booking, Payment, React UI, Dashboard và Docker Frontend.

1.3.1. Phạm vi trong đề tài

- Quản lý loại sân (field type) và sân thể thao (field) cụ thể theo từng loại.
- Quản lý đặt sân (booking) theo ngày và khung giờ, có kiểm tra trùng lịch.
- Quản lý thanh toán (payment) với hai hình thức: tiền mặt và chuyển khoản (kèm ảnh minh chứng).
- Quản lý người dùng và phân quyền theo 3 vai trò: Customer, Staff (hoặc Manager), Admin.
- Thống kê, báo cáo doanh thu và tình trạng booking theo khoảng thời gian.
- Xây dựng RESTful API bằng Node.js/Express, xác thực bằng JWT, tài liệu hóa bằng Swagger.
- Xây dựng giao diện web bằng ReactJS cho ba nhóm người dùng (Customer, Staff, Admin).
- Đóng gói và triển khai bằng Docker, Docker Compose.

1.3.2. Phạm vi ngoài đề tài

- Tích hợp cổng thanh toán trực tuyến thực tế (VNPay, MoMo, ZaloPay...); hệ thống hiện chỉ hỗ trợ xác nhận chuyển khoản thủ công qua ảnh minh chứng.
- Ứng dụng di động (mobile app) native cho iOS/Android.
- Gửi thông báo qua email/SMS khi trạng thái booking hoặc thanh toán thay đổi.
- Đánh giá, bình luận (review) của khách hàng đối với từng sân.

1.4. Đối tượng sử dụng

Hệ thống được thiết kế hướng tới 3 nhóm đối tượng sử dụng chính với các vai trò và quyền hạn được định nghĩa rõ ràng như sau:

Vai trò	Mô tả	Chức năng chính
Customer (Khách hàng)	Người dùng đăng ký tài khoản để đặt sân	Xem danh sách sân, tra cứu giờ trống, đặt sân, thực hiện thanh toán, xem lịch sử đặt sân cá nhân, hủy booking và cập nhật thông tin cá nhân.
Staff (Nhân viên)	Nhân viên vận hành cơ sở, do Admin tạo tài khoản	Quản lý sân/loại sân, theo dõi và cập nhật trạng thái đặt sân, duyệt thanh toán từ khách hàng, tạo và xem báo cáo thống kê.
Admin (Quản trị viên)	Người quản lý cao nhất của hệ thống	Toàn quyền như Staff, quản lý danh sách người dùng và nhân viên, xóa dữ liệu quan trọng như sân, booking, thanh toán nhằm đảm bảo tính an toàn dữ liệu.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Agile

Agile là phương pháp phát triển phần mềm theo hướng lặp và tăng trưởng. Thay vì thực hiện toàn bộ dự án theo một chuỗi công việc cố định (như mô hình thác nước Waterfall truyền thống), nhóm phát triển chia sản phẩm thành các phần tăng trưởng nhỏ hơn (increments) có thể hoàn thành và kiểm thử trong từng khoảng thời gian ngắn hạn. Cách tiếp cận này giúp nhóm phát triển thường xuyên nhận được phản hồi từ khách hàng, theo dõi tiến độ một cách linh hoạt và điều chỉnh kế hoạch kịp thời khi có thay đổi từ thực tế.

Nguyên lý cốt lõi của Agile dựa trên Tuyên ngôn Agile với 4 giá trị cốt lõi và 12 nguyên tắc chỉ đạo, nhấn mạnh vào sự tương tác giữa các cá nhân, sản phẩm chạy tốt, sự cộng tác với khách hàng và khả năng phản hồi linh hoạt trước các thay đổi.

2.2. Scrum

Scrum là một framework phát triển phần mềm phổ biến nhất trong triết lý Agile, tổ chức quá trình phát triển sản phẩm thành các chu kỳ lặp cố định gọi là các Sprint (thường kéo dài từ 1 đến 4 tuần). Mỗi Sprint đều có mục tiêu cụ thể, một danh sách công việc cần làm (Sprint Backlog) và một phần sản phẩm tăng trưởng có khả năng bàn giao.

Trong dự án này, Product Backlog được phân rã thành các Epic lớn và các Task chi tiết có gán vai trò người thực hiện (Assignee), độ phức tạp (Story Points). Các sự kiện của Scrum (như Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective) được mô phỏng linh hoạt thông qua việc cập nhật các trạng thái trên Scrum Board của Jira.

Các thành phần được sử dụng trong quá trình quản lý dự án gồm:

- Product Backlog: Danh sách tích lũy các yêu cầu của dự án, sắp xếp theo thứ tự ưu tiên.
- Sprint Backlog: Danh sách các công việc được lựa chọn từ Product Backlog để hoàn thành trong Sprint hiện tại.
- User Story: Các yêu cầu được viết dưới góc nhìn của người dùng cuối.
- Story Points: Đơn vị ước lượng độ phức tạp tương đối của các công việc.
- Burndown Chart: Biểu đồ trực quan hóa tiến độ công việc còn lại so với thời gian trong một Sprint.

- Scrum Board: Bảng theo dõi trạng thái công việc qua các cột: To Do, In Progress, In Review, Done.

2.3. Công nghệ sử dụng

Dự án áp dụng các công nghệ hiện đại, phổ biến trong công nghiệp phần mềm nhằm đảm bảo hiệu năng, tính mở rộng và khả năng đóng gói đồng nhất:

Thành phần / Công nghệ	Vai trò và giải pháp chi tiết trong hệ thống
NodeJS (Express 5)	Môi trường thực thi JavaScript phía server và framework xây dựng cấu trúc Backend RESTful API gọn nhẹ, hiệu năng cao.
MongoDB (Mongoose ODM)	Cơ sở dữ liệu NoSQL lưu trữ dạng JSON-like document linh hoạt, ánh xạ Schema bằng Mongoose để ràng buộc kiểu dữ liệu.
Bcrypt & JWT	Bcrypt băm mật khẩu người dùng lưu trong DB; jsonwebtoken sinh và xác thực token truy cập không trạng thái (stateless) có hạn 7 ngày.
React 19 & Vite	Xây dựng giao diện người dùng SPA (Single Page Application) mượt mà, build dự án cực nhanh với Vite dev-server.
React Router DOM	Xử lý định tuyến ứng dụng phía client, phân nhóm layout và cấu hình ProtectedRoute bảo vệ các trang theo quyền hạn.
Bootstrap 5	Thư viện CSS Responsive giúp giao diện hiển thị tối ưu trên nhiều thiết bị (Desktop, Tablet, Mobile).
Axios	HTTP client thực hiện các cuộc gọi API tới Backend, sử dụng Interceptor để tự động gắn token JWT vào Header Authorization.
Docker & Docker Compose	Đóng gói toàn bộ mã nguồn Frontend, Backend và Database đóng gói trong 3 container riêng biệt, chạy nhất quán chỉ với một dòng lệnh.

Thành phần / Công nghệ	Vai trò và giải pháp chi tiết trong hệ thống
Jira	Công cụ quản lý dự án theo mô hình Agile-Scrum, theo dõi tiến độ công việc, gán Story Points và Burndown Chart.
Git & GitHub	Hệ thống quản lý phiên bản mã nguồn, theo dõi lịch sử commit của từng thành viên trong nhóm.
Swagger UI (OpenAPI 3.0)	Tự động sinh tài liệu hóa API chi tiết tại đường dẫn /api-docs, hỗ trợ kiểm thử trực quan trên trình duyệt.

2.4. Kiến trúc ứng dụng

Hệ thống được thiết kế theo kiến trúc Client - Server tách biệt hoàn toàn giúp tối ưu hóa tải trọng và tăng tính độc lập khi phát triển phần mềm:

- Frontend: Xây dựng hoàn toàn bằng ReactJS hoạt động như một ứng dụng đơn trang (SPA) trên trình duyệt người dùng. Frontend chịu trách nhiệm dựng giao diện, quản lý trạng thái hiển thị, bắt lỗi nhập liệu (validate form) và giao tiếp với Backend thông qua các truy vấn HTTP (gửi và nhận dữ liệu dưới dạng JSON).
- Backend: Được xây dựng bằng Node.js/Express, hoạt động như một RESTful API server. Backend không can thiệp vào việc dựng giao diện (UI) mà chỉ tập trung xử lý nghiệp vụ, kiểm tra ràng buộc logic, kiểm tra phân quyền thông qua middleware, tương tác với Database và trả về cấu trúc dữ liệu JSON thống nhất. Tầng Backend được thiết kế theo mô hình phân lớp rõ rệt: Routes -> Middlewares -> Controllers -> Services -> Models.
- Database: Sử dụng MongoDB chạy trong container riêng. Mọi quan hệ giữa các thực thể được liên kết thông qua tham chiếu ObjectId hợp lý, giúp tối ưu hiệu năng đọc ghi dữ liệu NoSQL.

CHƯƠNG 3. XÁC ĐỊNH NHU CẦU

3.1. Đặc tả mục tiêu dự án

Mục tiêu của dự án là xây dựng hệ thống hỗ trợ quản lý sân thể thao từ khâu người dùng đăng nhập, xem sân, đặt sân, kiểm tra khung giờ, thanh toán cho đến quản trị sân và booking. Song song với mục tiêu sản phẩm, dự án phải tái hiện quy trình phát triển Agile – Scrum thông qua Product Backlog, Sprint Backlog, phân công trên Jira, commit mã nguồn và Burndown Chart. Điều này giúp nhóm thực hành kỹ năng làm việc thực tế, hiểu rõ cách vận hành của một nhóm phát triển phần mềm hiện đại trong công nghiệp.

3.2. Xác định Personas

Để định hình chính xác các yêu cầu hệ thống, nhóm xây dựng ba hồ sơ người dùng tiêu biểu (Personas) đại diện cho ba đối tượng tương tác trực tiếp với ứng dụng:

Persona / Vai trò	Mục tiêu / Nhu cầu chính	Chức năng hệ thống liên quan
Customer (Anh Minh, 22 tuổi, sinh viên)	Có nhu cầu tìm kiếm sân bóng trống vào tối thứ 7, đặt sân nhanh chóng, thanh toán chuyển khoản dễ dàng bằng điện thoại mà không cần gọi điện hỏi chủ sân.	Authentication (Đăng ký/Đăng nhập), Field (Xem/lọc sân), Availability (Tra cứu ô giờ trống trên lưới trực quan), Booking (Tạo đặt sân), Payment (Tải ảnh chụp minh chứng chuyển khoản).
Staff (Chị Hoa, 28 tuổi, nhân viên quầy)	Cần theo dõi toàn bộ lịch đặt sân trong ngày để sắp xếp sân cho khách, duyệt nhanh các giao dịch chuyển khoản khi có thông báo ảnh minh chứng, cập nhật trạng thái sân khi bảo trì.	Field Management (Cập nhật trạng thái hoạt động/bảo trì), Booking Management (Xác nhận/Hủy đơn), Payment Management (Duyệt/Từ chối thanh toán chuyển khoản), Dashboard thống kê hàng ngày.
Admin (Anh Hải, 35 tuổi, chủ cơ sở)	Cần theo dõi doanh thu tổng quan theo tuần, tháng để ra quyết định kinh doanh; quản lý phân quyền nhân viên quầy; có	User/Role Management (Tạo tài khoản Staff, xóa tài khoản vi phạm), Toàn quyền quản lý sân, booking, thanh toán, Thống kê

Persona / Vai trò	Mục tiêu / Nhu cầu chính	Chức năng hệ thống liên quan
	quyền xóa hoặc sửa các dữ liệu nhạy cảm để tránh thất thoát.	báo cáo doanh thu chi tiết lưu trong Reports.

3.3. Xây dựng User Stories

Các yêu cầu chức năng của hệ thống được cụ thể hóa bằng các User Stories chuẩn Agile, mô tả dưới góc nhìn mong muốn của tác nhân sử dụng:

Mã US	Vai trò tác nhân	Nội dung User Story chi tiết
US01	Customer	Là Customer, tôi muốn đăng ký tài khoản bằng số điện thoại để có thể đăng nhập sử dụng dịch vụ của hệ thống.
US02	Customer	Là Customer, tôi muốn đăng nhập bằng số điện thoại và mật khẩu để truy cập các chức năng đặt sân và xem lịch sử của riêng mình.
US03	Customer	Là Customer, tôi muốn xem danh sách sân và chi tiết từng sân (hình ảnh, giá thuê, vị trí) để dễ dàng lựa chọn sân phù hợp với nhu cầu.
US04	Customer	Là Customer, tôi muốn tra cứu lưới khung giờ trống của sân theo ngày để tránh chọn trùng với khung giờ người khác đã đặt.
US05	Customer	Là Customer, tôi muốn tạo đơn đặt sân (booking) trong các khung giờ trống và nhận trạng thái chờ thanh toán (pending).
US06	Customer	Là Customer, tôi muốn thực hiện thanh toán cho đơn đặt sân bằng cách tải lên ảnh chuyển khoản ngân hàng hoặc chọn trả tiền mặt tại quầy.
US07	Admin/Staff	Là Admin/Staff, tôi muốn quản lý thông tin các sân và loại sân (thêm, sửa, cập nhật trạng thái) để đảm bảo thông tin luôn chính xác.
US08	Admin/Staff	Là Admin/Staff, tôi muốn xem và cập nhật trạng thái các đơn đặt sân (xác nhận, hủy đơn, cập nhật lịch) để vận hành

Mã US	Vai trò tác nhân	Nội dung User Story chi tiết
		cơ sở hoạt động trơn tru.
US09	Chung	Là người dùng hệ thống, tôi muốn các API nhạy cảm được bảo vệ bằng JWT để đảm bảo an toàn thông tin cá nhân và quyền truy cập.
US10	Admin	Là Admin, tôi muốn xem Dashboard và các báo cáo doanh thu trực quan để kiểm soát tình hình kinh doanh của cơ sở.

3.4. Xác định Product Backlog

Từ các User Stories, nhóm phân rã thành ba Epic chính tương ứng với ba nhóm chức năng kỹ thuật, phân công chi tiết cho 3 thành viên như sau:

Mã Epic	Tên Epic lớn	Mô tả nhóm Task tiêu biểu	Thành viên phụ trách
E1	Authentication & User Management	Cấu hình base Backend, Swagger API docs, Schema User, Authentication Register/Login bằng Bcrypt & JWT, Middleware xác thực và kiểm soát quyền hạn phân lớp.	Thành viên 1 (Nguyễn Hữu Lộc)
E2	Core Booking Engine & Database	Thiết kế thực thể ERD, thiết kế Schema MongoDB cốt lõi (Field, FieldType, Booking, Payment), viết các API CRUD sâu, giải thuật tự động kiểm tra trùng lịch đặt sân và dọn dẹp	Thành viên 2 (Nguyễn Thị Hân Du)

Mã Epic	Tên Epic lớn	Mô tả nhóm Task tiêu biểu	Thành viên phụ trách
		booking hết hạn (lazy expiration).	
E3	React UI & Dashboard	Khởi tạo Vite + ReactJS, thiết kế định tuyến AppRoutes, xây dựng giao diện đăng ký/đăng nhập, giao diện xem danh sách và lưới đặt sân trực quan, giao diện duyệt thanh toán của nhân viên, Admin Dashboard thống kê.	Thành viên 3 (Nguyễn Gia Nguyễn)

3.5. Xác định nhu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ cốt lõi, hệ thống cần đáp ứng các tiêu chuẩn kỹ thuật nghiêm ngặt về chất lượng hệ thống (nhu cầu phi chức năng):

Loại yêu cầu	Mô tả tiêu chí chất lượng kỹ thuật chi tiết
Bảo mật (Security)	Mật khẩu người dùng bắt buộc được băm bằng bcrypt trước khi lưu; cơ chế xác thực không trạng thái JWT hết hạn sau 7 ngày; phân quyền truy cập route thông qua middleware authenticate/authorize; thiết lập header bảo mật chống tấn công XSS bằng helmet.
Hiệu năng (Performance)	Tạo chỉ mục duy nhất (unique index) trên các trường phone, email trong users để tăng tốc tìm kiếm và đăng nhập; truy vấn kiểm tra trùng lịch đặt sân được tối ưu bằng toán tử \$expr ngay trong Aggregation Pipeline của MongoDB nhằm giảm thời gian phản hồi dưới 100ms.
Khả năng bảo trì & mở rộng	Tổ chức mã nguồn Backend theo kiến trúc phân lớp rõ ràng (Route – Controller – Service – Model) giúp việc sửa đổi

Loại yêu cầu	Mô tả tiêu chí chất lượng kỹ thuật chi tiết
	logic nghiệp vụ trong tầng Service không làm ảnh hưởng tới luồng gọi API ở Controller và Route.
Khả năng triển khai	Đóng gói toàn bộ mã nguồn Backend, Frontend và Database MongoDB vào các Docker Image độc lập, đồng nhất cấu hình biến môi trường và chạy đồng thời thông qua Docker Compose.
Tài liệu hóa (Documentation)	Toàn bộ API được tự động viết tài liệu hóa bằng JSDoc chuẩn OpenAPI 3.0, tích hợp giao diện trực quan Swagger UI chạy tại địa chỉ /api-docs để hỗ trợ đội ngũ kiểm thử hoạt động hiệu quả.
Tính nhất quán dữ liệu	Khi một đơn đặt sân bị hủy (cancel) hoặc tự động hết hạn do quá thời gian chờ thanh toán, hệ thống tự động cập nhật trạng thái của bản ghi Payment pending liên quan sang failed để đảm bảo tính đồng bộ dữ liệu.
Trải nghiệm sử dụng (Usability)	Giao diện React xây dựng theo hướng responsive tương thích hiển thị mượt mà từ máy tính đến thiết bị di động, sử dụng SweetAlert2 để phản hồi thông báo trực quan, bắt lỗi (validate) ngay tại client.

3.6. Các tác nhân (Actor)

Hệ thống xác định ba tác nhân chính tham gia vận hành và tương tác trực tiếp là Customer, Staff và Admin. Chi tiết mối quan hệ của các tác nhân này đã được đặc tả tại Mục 1.4 của báo cáo.

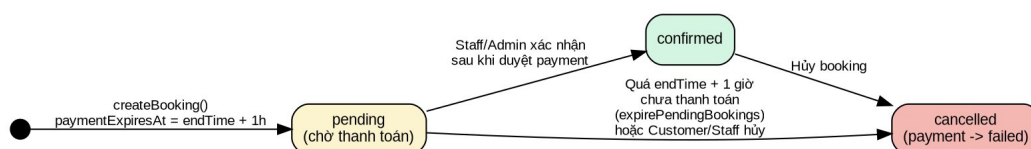
3.7. Đặc tả Use Case chi tiết

Dưới đây là đặc tả chi tiết của 7 Use Cases cốt lõi trong hệ thống quản lý vận hành sân thể thao:

UC-01: Đặt sân (Create Booking)

Tác nhân (Actor)	Customer
Luồng công việc	Khách hàng xem lưới khung giờ trống/đã đặt của sân theo ngày

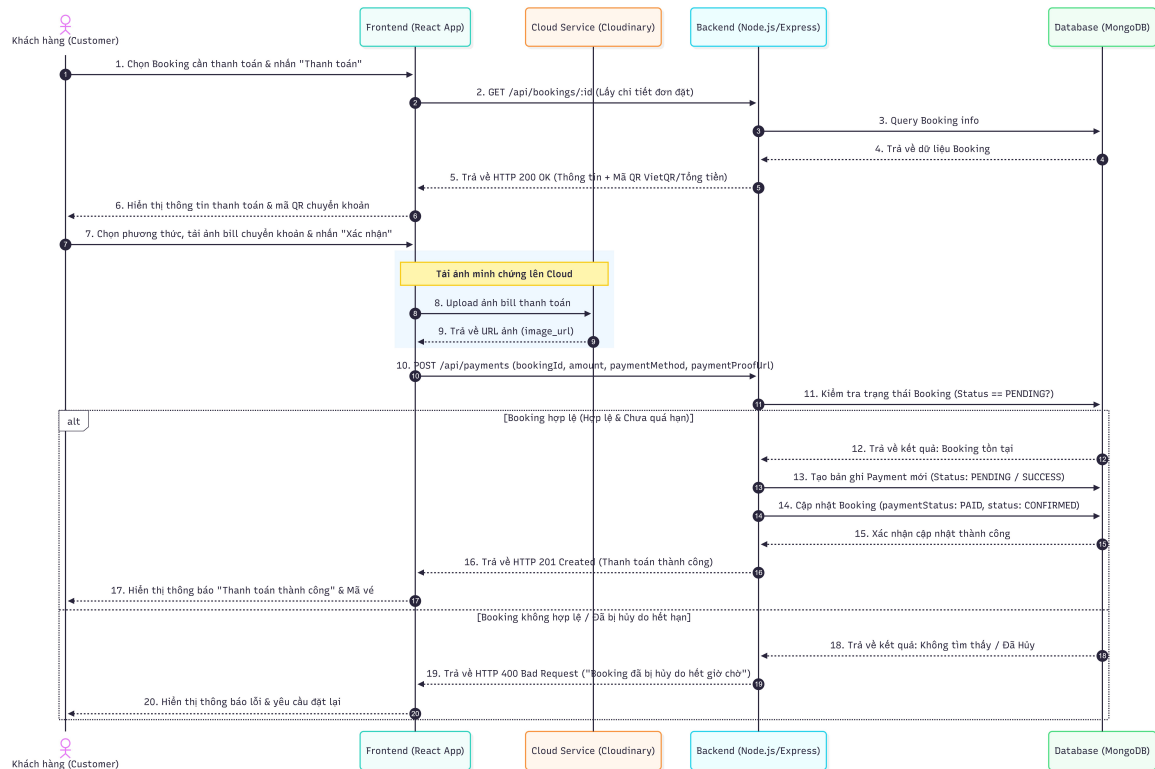
chính	(GET /api/bookings/availability) -> Chọn một hoặc nhiều ô giờ liên tiếp (mỗi ô = 1 giờ, khung hoạt động 06:00–21:00) -> Hệ thống kiểm tra sân đang hoạt động (active) -> Hệ thống kiểm tra khung giờ không trùng lịch đặt sân khác (trạng thái khác cancelled) -> Hệ thống tự động tính toán tổng tiền = số giờ x giá thuê/giờ -> Tạo đơn đặt sân trạng thái pending, tính hạn thanh toán paymentExpiresAt = giờ kết thúc + 1 giờ (theo múi giờ Việt Nam UTC+07:00).
Luồng ngoại lệ	Sân không tồn tại hoặc đang bảo trì (maintenance) -> Báo lỗi. Khung giờ chọn bị trùng lịch đặt sân khác -> Báo lỗi 'Khung giờ này bị trùng với một booking khác'. Giờ kết thúc nhỏ hơn hoặc bằng giờ bắt đầu -> Báo lỗi định dạng thời gian.
Kết quả đạt được	Dữ liệu được cập nhật đồng bộ, đảm bảo tính đúng đắn nghiệp vụ.



Hình 3.1 Sơ đồ chuyển trạng thái của Booking

UC-02: Thanh toán booking (Create Payment)

Tác nhân (Actor)	Customer
Luồng công việc chính	Khách hàng chọn phương thức thanh toán (cash hoặc bank_transfer) cho đơn đặt sân đang ở trạng thái pending -> Nếu là chuyển khoản (bank_transfer), khách hàng tải lên hình ảnh chụp màn hình minh chứng giao dịch chuyển khoản thành công -> Hệ thống kiểm tra và tạo bản ghi Payment ở trạng thái pending, lưu đường dẫn ảnh.
Luồng ngoại lệ	Đơn đặt sân đã bị hủy trước đó -> Không cho thanh toán. Đơn đặt sân không ở trạng thái pending -> Báo lỗi. Khách hàng chọn bank_transfer nhưng không tải ảnh lên -> Báo lỗi 'Vui lòng tải ảnh xác nhận chuyển khoản'.
Kết quả đạt được	Dữ liệu được cập nhật đồng bộ, đảm bảo tính đúng đắn nghiệp vụ.



Hình 3.2 Sơ đồ tuần tự cho luồng thanh toán

UC-03: Xác nhận / cập nhật booking

Tác nhân (Actor)	Staff, Admin
Luồng công việc chính	Nhân viên quầy hoặc Admin chọn đơn đặt sân cần cập nhật thông tin trên giao diện quản lý -> Thực hiện điều chỉnh thông tin sân, ngày đặt, hoặc khung giờ đặt (nếu khách hàng yêu cầu đổi lịch) -> Hệ thống tự động kiểm tra lại trùng lịch (loại trừ chính đơn đang sửa) -> Tính lại tổng tiền -> Lưu thay đổi vào database.
Luồng ngoại lệ	Khung giờ mới cập nhật bị trùng với một đơn đặt sân khác đang hoạt động -> Báo lỗi và không lưu dữ liệu.
Kết quả đạt được	Dữ liệu được cập nhật đồng bộ, đảm bảo tính đúng đắn nghiệp vụ.

UC-04: Hủy booking (Cancel Booking)

Tác nhân (Actor)	Customer (đơn của mình), Staff, Admin
Luồng công việc chính	Người dùng chọn hủy đơn đặt sân đang hoạt động -> Hệ thống chuyển trạng thái của đơn đặt sân sang cancelled, giải phóng khung giờ, đặt lại hạn giữ sân paymentExpiresAt = null -> Nếu đơn đặt

	sân đó có bản ghi Payment đang ở trạng thái pending liên quan, hệ thống tự động chuyển trạng thái thanh toán sang failed để đồng nhất dữ liệu.
Luồng ngoại lệ	Đơn đặt sân đã ở trạng thái cancelled trước đó -> Báo lỗi 'Booking đã được hủy trước đó'.
Kết quả đạt được	Dữ liệu được cập nhật đồng bộ, đảm bảo tính đúng đắn nghiệp vụ.

UC-05: Tự động hết hạn booking chờ thanh toán

Tác nhân (Actor)	Hệ thống (Tự động kích hoạt khi có truy vấn)
Luồng công việc chính	Mỗi khi có yêu cầu truy vấn danh sách đặt sân hoặc tra cứu khung giờ trống, hệ thống tự động chạy kiểm tra dọn dẹp (lazy expiration) -> Hệ thống tìm các đơn đặt sân đang ở trạng thái pending có thời điểm hiện tại lớn hơn hoặc bằng hạn thanh toán paymentExpiresAt -> Chuyển trạng thái đơn đặt sân đó sang cancelled, đặt paymentExpiresAt = null -> Đồng bộ trạng thái thanh toán pending liên quan sang failed để giải phóng khung giờ.
Luồng ngoại lệ	Các đơn đặt sân pending có hạn thanh toán chưa tới (trong tương lai) -> Giữ nguyên trạng thái pending, không thực hiện hủy.
Kết quả đạt được	Dữ liệu được cập nhật đồng bộ, đảm bảo tính đúng đắn nghiệp vụ.

UC-06: Thống kê báo cáo doanh thu

Tác nhân (Actor)	Staff, Admin
Luồng công việc chính	Người dùng đăng nhập quyền Staff/Admin truy cập trang thống kê, nhập khoảng thời gian cần thống kê (fromDate, toDate) -> Hệ thống thực hiện truy vấn và tính toán doanh thu từ các giao dịch thanh toán thành công (paid) và số lượng đặt sân theo từng trạng thái -> Trả về tổng doanh thu, tổng số đặt sân, số lượng chi tiết và thống kê doanh thu chi tiết theo từng ngày (dailyStats) -> Lưu báo cáo vào reports.
Luồng ngoại lệ	Người dùng không truyền fromDate hoặc toDate -> Báo lỗi 'Ngày

	không được để trống'. Nhập ngày sai định dạng hoặc fromDate lớn hơn toDate -> Báo lỗi.
Kết quả đạt được	Dữ liệu được cập nhật đồng bộ, đảm bảo tính đúng đắn nghiệp vụ.

UC-07: Tra cứu khung giờ đã đặt (Availability)

Tác nhân (Actor)	Customer, Staff, Admin
Luồng công việc chính	Giao diện gửi yêu cầu tra cứu kèm mã sân (fieldId) và ngày đặt (bookingDate) -> Hệ thống tự động gọi hàm dọn dẹp hết hạn để đảm bảo dữ liệu mới nhất -> Truy vấn toàn bộ đơn đặt sân của sân đó trong ngày có trạng thái khác cancelled -> Trả về danh sách đơn rút gọn (startTime, endTime, status) -> Giao diện dựng lưới trực quan và tô màu đỏ các khung giờ đã bị đặt.
Luồng ngoại lệ	Thiếu tham số fieldId hoặc bookingDate -> Báo lỗi yêu cầu nhập đủ dữ liệu.
Kết quả đạt được	Dữ liệu được cập nhật đồng bộ, đảm bảo tính đúng đắn nghiệp vụ.

CHƯƠNG 4. LẬP KẾ HOẠCH

HIỆN THỰC HÓA BẰNG SCRUM

4.1. Nguyên tắc sắp xếp ưu tiên Product Backlog

Thứ tự ưu tiên phát triển các Epic và Task trong Product Backlog được đội ngũ phát triển phân tích kỹ lưỡng dựa trên sự phụ thuộc logic kỹ thuật giữa các chức năng và giá trị nghiệp vụ của chúng mang lại:

1. Sprint 1 - Foundation & Authentication (Dựng Khung & Đăng Nhập): Được xếp ưu tiên cao nhất nhằm khởi tạo Base Code cho toàn bộ hệ thống, thiết lập Database lõi, xử lý xác thực tài khoản (JWT/Role) end-to-end từ giao diện React xuống Backend và hoàn thành tập API/Giao diện quản lý danh sách sân.
2. Sprint 2 - Core Booking & Field Management (Nghiệp Vụ Đặt Sân): Tập trung vào nghiệp vụ cốt lõi của ứng dụng. Nhóm triển khai Middleware bảo mật API, thuật toán kiểm tra chồng giờ, giao diện chọn khung giờ trực quan cho khách hàng và tính năng quản trị sân bóng dành cho Admin.
3. Sprint 3 - Payment, Dashboard & Integration (Thanh Toán & Hoàn Thiện): Tích hợp cổng thanh toán trực tuyến, xây dựng Admin Dashboard thống kê báo cáo, tiến hành kiểm thử toàn diện end-to-end và đóng gói hệ thống đa container với Docker.

4.2. Phân chia Sprint

Quá trình phát triển được phân chia thành 3 Sprint cụ thể với mục tiêu rõ ràng và kết quả đầu ra đo lường được:

Sprint	Nội dung nghiệp vụ chính	Kết quả đầu ra kỳ vọng (DoD)
Sprint 1: Foundation & Authentication	Setup Base Backend (Swagger), khởi tạo DB Schema cốt lõi, API Auth & User/Role, API Quản lý sân, Setup React SPA & Docker Frontend, UI Auth/Route Guard và UI Danh sách sân.	Hệ thống có khung chạy ổn định; đăng ký/đăng nhập hoạt động mượt mà end-to-end từ UI đến Backend; người dùng xem được danh sách/chi tiết sân qua giao diện React.
Sprint 2: Core	Phân quyền Middleware API, thuật toán & API kiểm tra	Admin thêm/sửa/xóa sân trực quan trên UI; khách hàng xem được lưới

Sprint	Nội dung nghiệp vụ chính	Kết quả đầu ra kỳ vọng (DoD)
Booking & Field Management	trùng lịch, API Đặt sân & quản lý trạng thái, UI chọn khung giờ & Form đặt sân, UI Admin quản lý sân.	khung giờ trống thực tế và gửi yêu cầu đặt sân thành công vượt qua kiểm thử trùng lịch.
Sprint 3: Payment, Dashboard & Integration	Tích hợp cổng thanh toán trực tuyến (VNPay/MoMo/QR Code), UI Admin quản lý Booking & Dashboard thống kê, kiểm thử luồng end-to-end, tối ưu DB & hoàn thiện Docker.	Hoàn thiện luồng thanh toán đơn đặt sân; Admin theo dõi được doanh thu/thống kê trên Dashboard; toàn bộ hệ thống đóng gói chạy ổn định bằng Docker Compose.

4.3. Sprint Backlog và phân công

- Sprint 1:
 - Thành viên 1: Task 1.1 (Setup Base Backend & Swagger), Task 1.2 (Module User/Role DB), Task 1.3 (API Auth & JWT).
 - Thành viên 2: Task 2.1 (Schema DB cốt lõi), Task 2.2 (API CRUD Field & FieldType).
 - Thành viên 3: Task 3.1 (Setup React Project & Docker Frontend), Task 3.2 (UI Auth & Route Guard), Task 3.3 (UI Khách hàng - Danh sách & Chi tiết sân).
- Sprint 2:
 - Thành viên 1: Task 1.4 (Middleware xác thực Token & Phân quyền Route).
 - Thành viên 2: Task 2.3 (Thuật toán & API Kiểm tra trùng lịch), Task 2.4 (API Đặt sân & Quản lý trạng thái đơn).
 - Thành viên 3: Task 3.4 (UI Chọn khung giờ trống & Form Đặt sân), Task 3.5 (UI Admin - Quản lý Sân).
- Sprint 3:
 - Thành viên 1: Hỗ trợ kiểm thử API, tối ưu hóa câu lệnh DB & hoàn thiện tài liệu Swagger.

- Thành viên 2: Task 2.5 (Tích hợp API Cổng thanh toán VNPay / QR Code).
- Thành viên 3: Task 3.6 (UI Admin - Quản lý Booking & Dashboard Thống kê), Task 3.7 (Integration - Kiểm thử luồng Đặt sân -> Thanh toán -> Trạng thái & Tối ưu Docker).

4.4. Ước lượng Story Points

Các Task công việc được nhóm ước lượng độ phức tạp tương đối bằng phương pháp Planning Poker sử dụng dãy số Fibonacci tiêu chuẩn (1, 2, 3, 5, 8, 13) làm điểm Story Points như sau:

Mã Task	Mô tả công việc chi tiết	Thành viên phụ trách	Story Points
Task 1.1	Setup Base Backend & Swagger Docs	Thành viên 1	3
Task 1.2	Module User/Role DB & Seed data mẫu	Thành viên 1	3
Task 1.3	API Auth (Login/Register) & JWT	Thành viên 1	5
Task 2.1	Thiết kế & Khởi tạo Schema Database cốt lõi	Thành viên 2	5
Task 2.2	API Quản lý Sân (CRUD Field/FieldType)	Thành viên 2	5
Task 3.1	Setup React Project & Docker Frontend	Thành viên 3	5
Task 3.2	UI Auth & Route Guard (Nối API Auth từ TV1)	Thành viên 3	3
Task 3.3	UI Khách hàng - Danh sách & Chi tiết sân (Nối API từ TV2)	Thành viên 3	5
Task 1.4	Middleware xác thực Token & Phân quyền Route	Thành viên 1	5
Task 2.3	Thuật toán & API Kiểm tra trùng lịch đặt sân	Thành viên 2	8
Task 2.4	API Đặt sân (Booking) & Quản lý trạng thái đơn	Thành viên 2	8
Task 3.4	UI Chọn khung giờ trống & Form Đặt sân (Nối API từ TV2)	Thành viên 3	8

Mã Task	Mô tả công việc chi tiết	Thành viên phụ trách	Story Points
Task 3.5	UI Admin - Quản lý Sân (CRUD Field)	Thành viên 3	5
Task 2.5	Tích hợp API Cổng thanh toán (VNPay / MoMo / QR Code)	Thành viên 2	8
Task 3.6	UI Admin - Quản lý Booking & Dashboard Thống kê	Thành viên 3	8
Task 3.7	Integration - Kiểm thử luồng End-to-End & Tối ưu Docker	Thành viên 3	5

4.5. Lập kế hoạch và quản lý Sprint trên Jira

Toàn bộ vòng đời phát triển dự án được quản trị tập trung trên Jira Software theo mô hình Scrum Board chuyên nghiệp:

1. Thiết lập dự án: Nhóm khởi tạo một dự án dạng Company-managed software trên Jira, lựa chọn mẫu Scrum và kích hoạt các tính năng Roadmap, Backlog, Board, Reports.

2. Xây dựng Backlog: Đội ngũ nhập toàn bộ các User Stories và Epic vào Product Backlog, sau đó tiến hành phân rã thành các Task công việc cụ thể kèm mô tả chi tiết, tiêu chí nghiệm thu (DoD) và độ phức tạp Story Points.

3. Khởi tạo Sprint: Nhóm kéo các Task ưu tiên từ Product Backlog vào Sprint 1, gán Assignee và nhấn Start Sprint, thiết lập độ dài Sprint cố định.

4. Vận hành hàng ngày: Các thành viên thực hiện cập nhật tiến độ công việc hàng ngày bằng cách kéo thả các Task qua các cột trạng thái To Do -> In Progress -> In Review -> Done.

Spaces

Sport_ManagementSport_System1 ...

Summary List Board Backlog Development Forms Timeline Docs Reports +

Ask AI Search work Filter 2 Group

Work	Assignee	Reporter	Priority	Status
☐ SPOR-1 Authentication & User Management	HN Huu loc Nguyen	GN Gia Nguyen	Medium	Done ✓
☐ SPOR-6 Core Booking Engine & Database	Unassigned	GN Gia Nguyen	Medium	Done ✓
☐ SPOR-13 React UI & Dashboard	GN Gia Nguyen	GN Gia Nguyen	Medium	Done ✓
☒ SPOR-19 Task 3.6: Giao diện Admin - Quản lý Booking & D...	GN Gia Nguyen	GN Gia Nguyen	Medium	Done ✓
☒ SPOR-12 Task 2.5: Tích hợp API Cổng thanh toán (Payment)	H handu	GN Gia Nguyen	Medium	Done ✓
☒ SPOR-18 Task 3.5: Giao diện Admin - Quản lý Sân (CRUD)	GN Gia Nguyen	GN Gia Nguyen	Medium	Done ✓
☒ SPOR-17 Task 3.4: Giao diện Đặt sân & Chọn thanh toán	GN Gia Nguyen	GN Gia Nguyen	Medium	Done ✓
☒ SPOR-11 Task 2.4: Xây dựng API Đặt sân (Booking) & Quà...	H handu	GN Gia Nguyen	Medium	Done ✓
☒ SPOR-10 Task 2.3: Thuật toán & API Kiểm tra trùng lịch đặ...	H handu	GN Gia Nguyen	Medium	Done ✓

+ Create 18 of 18

Hình 4.1 Scrum Board trên Jira

5. Theo dõi tiến độ: Burndown Chart được Jira tự động cập nhật dựa trên số lượng Story Points đã hoàn thành của các Task chuyển sang trạng thái Done, giúp nhóm theo dõi trực quan tiến độ thực tế so với đường tiến độ lý tưởng.

CHƯƠNG 5. HIỆN THỰC HÓA KẾ HOẠCH

5.1. Thiết kế kiến trúc ứng dụng & Thư mục mã nguồn

Tầng Backend được tổ chức bài bản theo kiến trúc phân lớp (Layered Architecture) giúp tách biệt trách nhiệm rõ ràng, thuận tiện cho việc phát triển độc lập và bảo trì mã nguồn lâu dài. Cấu trúc thư mục của dự án được thiết lập đồng bộ như sau:

```
backend/src/
├── app.js          # Cấu hình Express app, đăng ký middleware & route
├── server.js       # Điểm khởi chạy, kết nối MongoDB, lắng nghe cổng
├── configs/
│   └── database.js # Cấu hình kết nối cơ sở dữ liệu MongoDB bằng
Mongoose
├── controllers/    # auth, field, fieldType, booking, payment, report,
user controllers
├── services/       # Chứa toàn bộ business logic nghiệp vụ của hệ thống
├── models/         # Định nghĩa các Schema Mongoose: User, Field,
FieldType, Booking, Payment, Report
├── routes/         # Định nghĩa endpoint RESTful API cho từng module
├── middlewares/    # auth.middleware.js, upload.js,
error.middleware.js
├── swagger/        # Định nghĩa JSDoc đặc tả API chuẩn OpenAPI 3.0
├── scripts/createAdmin.js # Script tự động tạo tài khoản Admin đầu tiên
của hệ thống
└── uploads/        # Thư mục lưu trữ hình ảnh sân và ảnh minh chứng
chuyển khoản
```

5.2. Thiết kế Database

Dữ liệu được lưu trữ trong MongoDB với cấu trúc 6 collection cốt lõi, được định nghĩa chặt chẽ bằng Mongoose Schemas đầy đủ như dưới đây để phục vụ cho các nghiệp vụ chính:

a) Model User (Lưu trữ người dùng và vai trò)

```
const userSchema = new mongoose.Schema({
  fullName: { type: String, required: true },
  phone: { type: String, required: true, unique: true },
  email: { type: String, unique: true, sparse: true },
  password: { type: String, required: true }, // Mật khẩu đã hash bằng bcrypt
  role: { type: String, enum: ['customer', 'staff', 'admin'], default:
'customer' }
}, { timestamps: true });
```

b) Model FieldType (Loại sân thể thao)

```
const fieldTypeSchema = new mongoose.Schema({
  name: { type: String, required: true },
  description: { type: String }
}, { timestamps: true });
```

c) Model Field (Sân thể thao cụ thể)

```
const fieldSchema = new mongoose.Schema({
  fieldName: { type: String, required: true },
  fieldTypeId: { type: ObjectId, ref: 'FieldType', required: true },
  location: { type: String },
  pricePerHour: { type: Number },
  image: { type: String, default: "" },
  description: { type: String, default: "" },
  rating: { type: Number, default: 0 },
  status: { type: String, enum: ['active', 'maintenance'], default: 'active' }
}, { timestamps: true });
```

d) Model Booking (Đơn đặt sân)

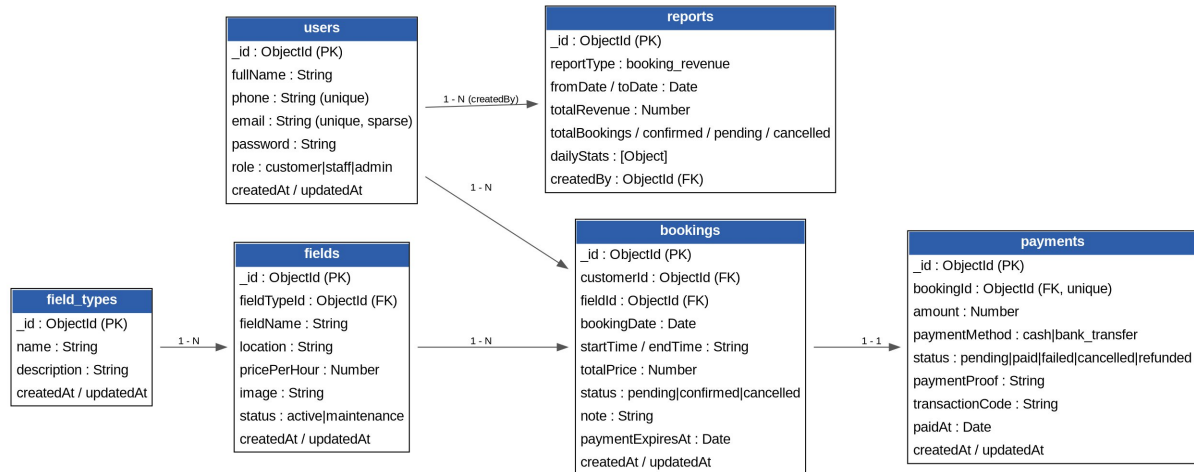
```
const bookingSchema = new mongoose.Schema({
  customerId: { type: ObjectId, ref: 'User', required: true },
  fieldId: { type: ObjectId, ref: 'Field', required: true },
  bookingDate: { type: Date, required: true },
  startTime: { type: String, required: true }, // Định dạng 'HH:mm'
  endTime: { type: String, required: true },
  totalPrice: { type: Number, default: 0 },
  status: { type: String, enum: ['pending', 'confirmed', 'cancelled'], default:
'pending' },
  note: { type: String, default: '' },
  paymentExpiresAt: { type: Date, default: null } // Hạn thanh toán: endTime
+ 1 giờ
}, { timestamps: true });
```

e) Model Payment (Thông tin thanh toán)

```
const paymentSchema = new mongoose.Schema({
  bookingId: { type: ObjectId, ref: 'Booking', required: true, unique: true },
  amount: { type: Number, required: true, min: 0 },
  paymentMethod: { type: String, enum: ['cash', 'bank_transfer'], required:
true },
  status: { type: String, enum: ['pending', 'paid', 'failed', 'cancelled',
'refunded'], default: 'pending' },
  paymentProof: { type: String, default: '' }, // Đường dẫn lưu trữ ảnh
chuyển khoản
  transactionCode: { type: String, default: '' },
  paidAt: { type: Date, default: null }
}, { timestamps: true });
```

f) Sơ đồ thực thể quan hệ ERD

Mối quan hệ giữa các thực thể được thiết lập chặt chẽ thông qua tham chiếu ObjectId logic ở tầng Mongoose ODM. Một thực thể loại sân (FieldType) liên kết một-nhiều tới các sân cụ thể (Field). Một khách hàng (User) có thể tạo nhiều đơn đặt sân (Booking). Mỗi đơn đặt sân (Booking) có quan hệ một-một duy nhất tới một thông tin thanh toán (Payment) nhờ ràng buộc unique đặt trên trường bookingId của Payment. Thông qua đó, tính toàn vẹn dữ liệu được đảm bảo an toàn.



Hình 5.1 Sơ đồ ERD

5.3. Thiết kế REST API & Phân quyền JWT

Hệ thống cung cấp hơn 25 endpoints API chuẩn RESTful, sử dụng Bearer Token JWT để bảo mật ở tầng middleware. Dưới đây là mã nguồn middleware xác thực và phân quyền RBAC được triển khai trong dự án:

```
// middlewares/auth.middleware.js
const jwt = require('jsonwebtoken');

const authenticate = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ success: false, message: 'Access token is missing' });
  }
  try {
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

```

    req.user = decoded; // Gán payload { id, role } vào request
    next();
  } catch (error) {
    return res.status(401).json({ success: false, message: 'Invalid or expired token' });
  }
};

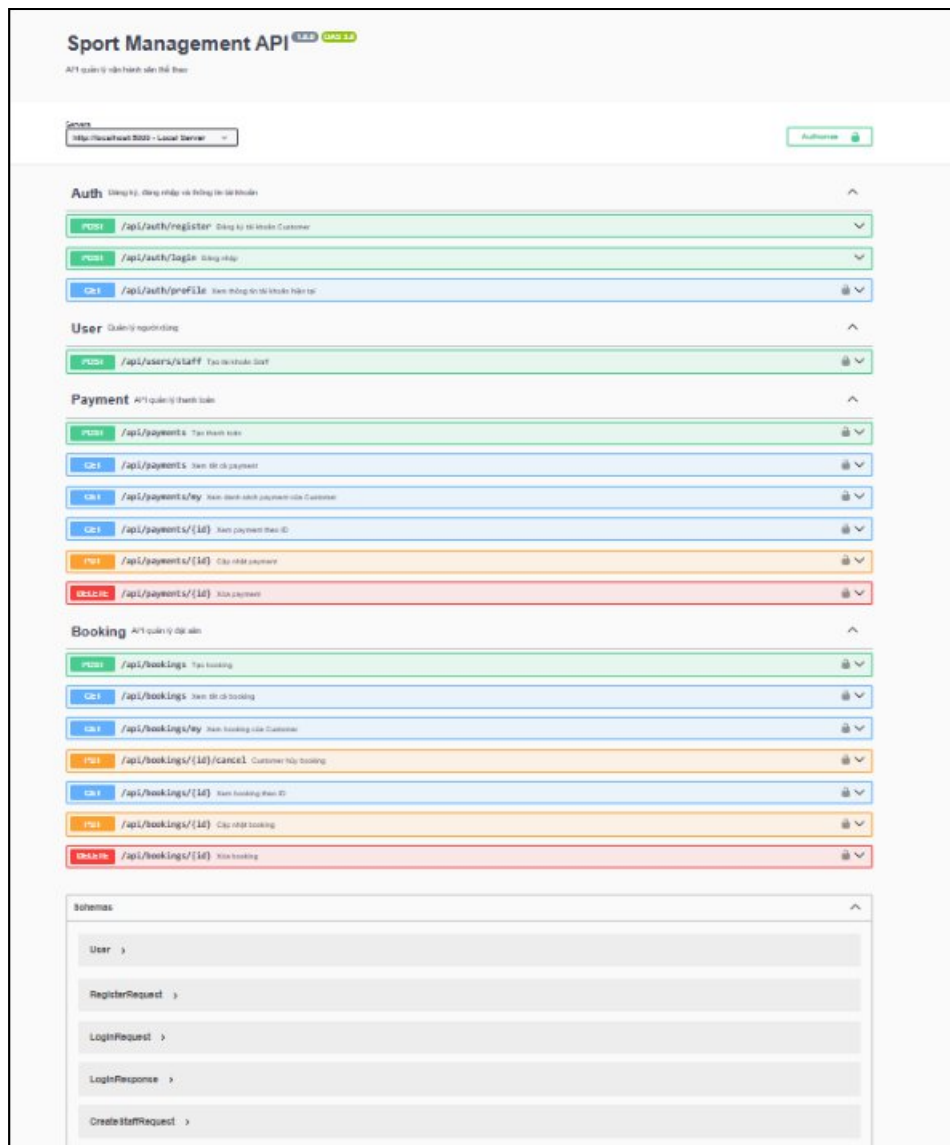
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ success: false, message:
'Unauthorized' });
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ success: false, message: 'Forbidden: Access
denied' });
    }
    next();
  };
};

```

5.4. Hiện thực các Sprint

5.4.1. Sprint 1 – Authentication & User Management

Thành viên 1 đã cấu hình base project Backend sử dụng Express 5 và Swagger để tài liệu hóa các endpoint API. Module xác thực hỗ trợ đăng ký tài khoản với mật khẩu được băm an toàn bằng Bcrypt (salt rounds = 10). Khi đăng nhập thành công, hệ thống phát hành một mã thông báo JWT chứa thông tin id và role của người dùng với thời hạn hiệu lực 7 ngày, giúp duy trì phiên đăng nhập không trạng thái (stateless) cực kỳ bảo mật.



Hình 5.2 Các endpoint API đã được tài liệu hóa thành công

5.4.2. Sprint 2 – Core Database, Field Management, Booking & Payment

Core Database, Field Management

Thành viên 2 đã xây dựng thành công database và các API quản lý loại sân, sân thể thao. Đặc biệt, để giải quyết triệt để bài toán trùng lịch đặt sân, dự án đã triển khai thuật toán kiểm tra chồng giờ tối ưu sử dụng toán tử \$expr ngay trong Aggregation Pipeline của MongoDB. Thuật toán hoạt động bằng cách chuyển đổi chuỗi giờ sang số thực (ví dụ: '18:30' -> 18.5) và so khớp điều kiện giao thoa thời gian đối với các đặt sân đang hoạt động:

```
// Kiểm tra trùng lịch đặt sân bằng MongoDB Aggregation
const overlappingBookings = await Booking.find({
  fieldId: fieldId,
```

```

    bookingDate: new Date(bookingDate),
    status: { $ne: 'cancelled' },
    $expr: {
      $and: [
        { $lt: [ { $toDouble: { $replaceOne: { input: '$startTime', find: ':',
replacement: '.' } } }, endTimeNum ] },
        { $gt: [ { $toDouble: { $replaceOne: { input: '$endTime', find: ':',
replacement: '.' } } }, startTimeNum ] }
      ]
    }
  });

```

Nếu mảng kết quả overlappingBookings trả về có độ dài lớn hơn 0, hệ thống sẽ lập tức từ chối yêu cầu đặt sân và phản hồi mã lỗi 400 kèm thông điệp báo trùng lịch chi tiết.

Booking & Payment

Trong Sprint này, cơ chế giới hạn thời gian giữ sân (giữ chỗ) của các booking pending đã được cải tiến thực tế hơn rất nhiều so với phiên bản cũ (vốn chỉ giữ 15 phút cố định kể từ lúc đặt). Hạn thanh toán mới được tính dựa trên khung giờ kết thúc của đơn chơi: $\text{paymentExpiresAt} = (\text{ngày đặt sân}) + (\text{giờ kết thúc endTime}) + 1 \text{ giờ}$, tính theo múi giờ Việt Nam (UTC+07:00). Cơ chế lazy expiration dọn dẹp các booking quá hạn tự động được gọi ở đầu mỗi truy vấn giúp hệ thống luôn hoạt động chính xác mà không cần một dịch vụ chạy nền tốn tài nguyên.

```

// Tự động dọn dẹp các booking pending quá hạn thanh toán (lazy
expiration)
const expirePendingBookings = async () => {
  const now = new Date();
  const expiredBookings = await Booking.find({
    status: 'pending',
    paymentExpiresAt: { $lte: now }
  });
};

```

```

for (const booking of expiredBookings) {
  booking.status = 'cancelled';
  booking.paymentExpiresAt = null;
  await booking.save();

  // Đồng bộ trạng thái thanh toán pending liên quan sang failed
  await Payment.findOneAndUpdate(
    { bookingId: booking._id, status: 'pending' },
    { status: 'failed' }
  );
}
};

```

5.4.3. Sprint 3 – React UI, Dashboard & Docker

Thành viên 3 đã xây dựng hoàn chỉnh giao diện ReactJS sử dụng Bootstrap 5 responsive. Điểm nhấn lớn nhất là giao diện lưới chọn khung giờ đặt sân (Booking Schedule) từ 06:00 đến 21:00 được cập nhật tự động sau mỗi 30 giây bằng setInterval giúp khách hàng lựa chọn trực quan và khóa các khung giờ đã qua trong ngày.

Hình 5.3 Giao diện lưới khung giờ đặt sân

Ngoài ra, tệp docker-compose.yml đã được cấu hình hoàn chỉnh để liên kết 3 containers frontend, backend và database chạy trơn tru trong cùng một mạng nội bộ:

```
# docker-compose.yml
version: '3.8'
services:
  mongodb:
    image: mongo:8.0
    ports:
      - "27018:27017"
    volumes:
      - mongodb_data:/data/db
    networks:
      - sport-network

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - mongodb
    environment:
      MONGO_URI: mongodb://mongodb:27017/sport_management
      JWT_SECRET: your_secret_key
    networks:
      - sport-network

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
```

- backend

environment:

- VITE_API_URL=http://localhost:5000/api

networks:

- sport-network

networks:

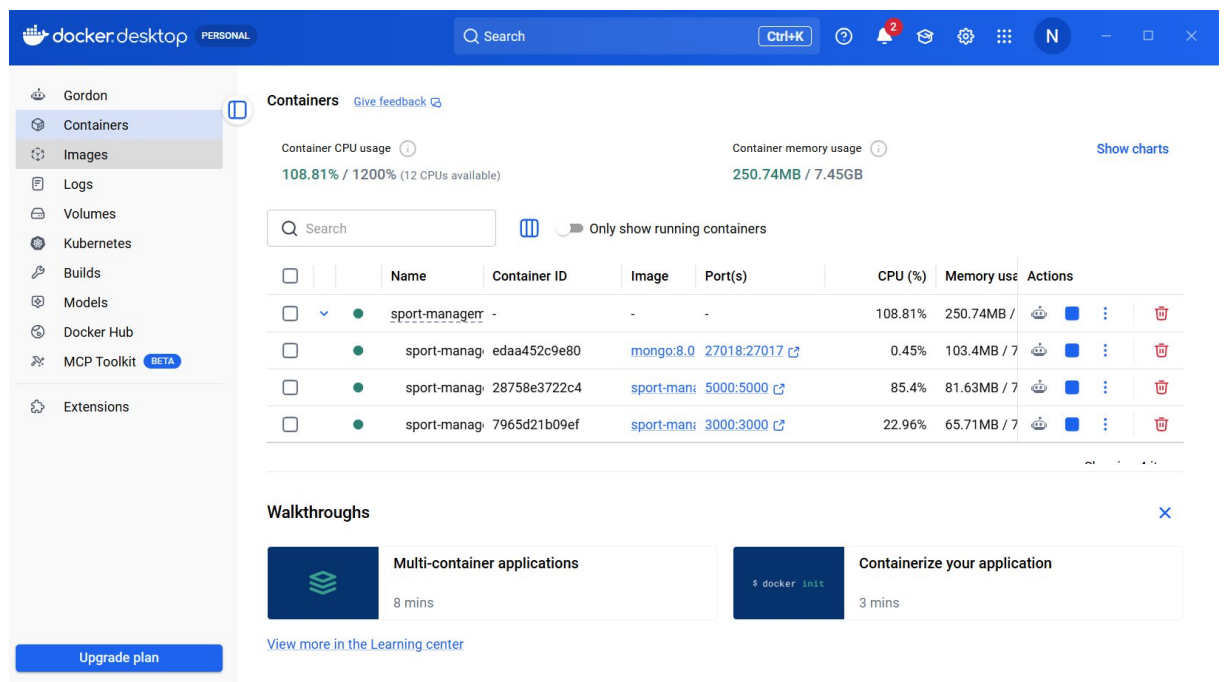
sport-network:

driver: bridge

volumes:

mongodb_data:

driver: local

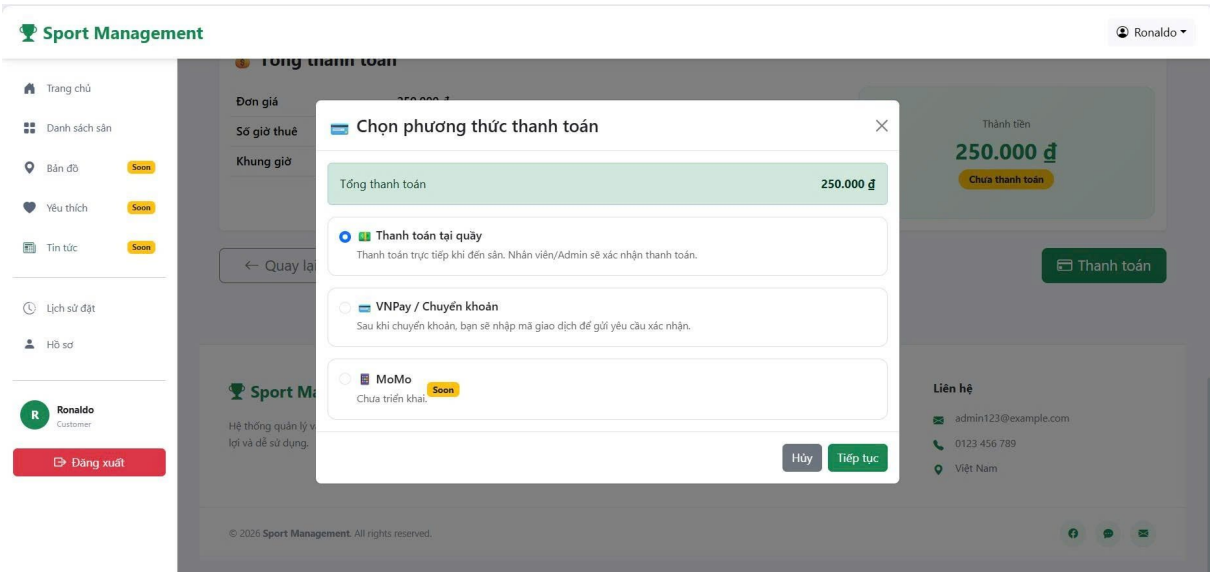


Hình 5.4 Chạy hệ thống trên Docker

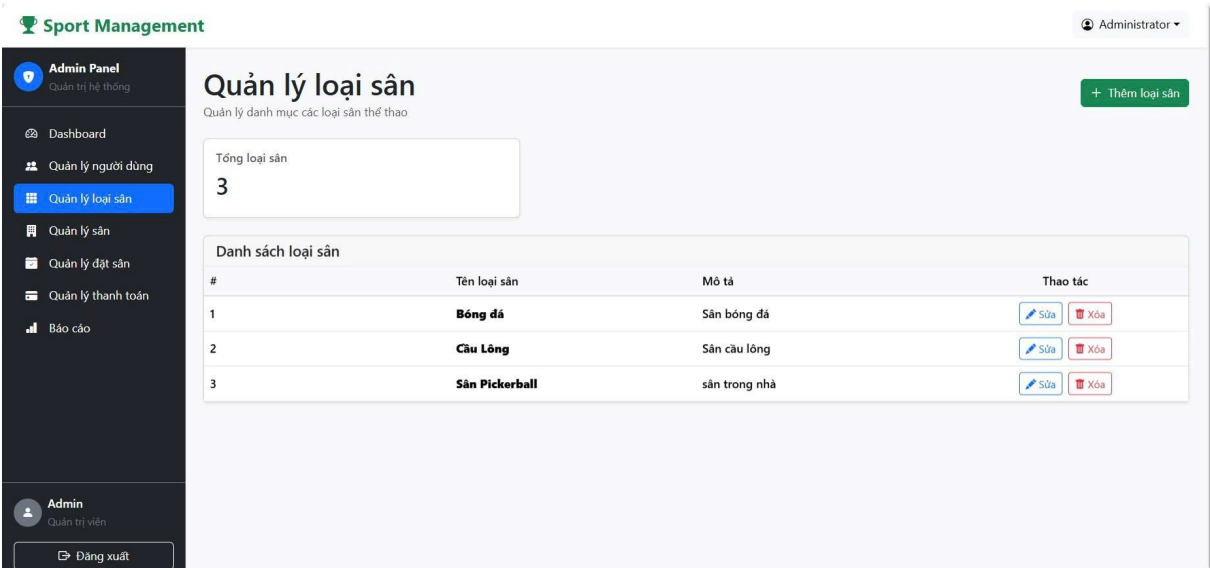
5.5. Xây dựng giao diện bằng mã nguồn HTML/CSS, quản lý mã nguồn Git/GitHub

Xây dựng giao diện bằng mã nguồn HTML/CSS Nhóm quyết định xây dựng trực tiếp giao diện người dùng bằng mã nguồn HTML5, CSS3 (*kết hợp framework Bootstrap / Tailwind / React components*). Phương pháp này giúp rút ngắn thời gian từ

ý tưởng đến sản phẩm thực tế, cho phép kiểm thử trực tiếp độ tương thích và phản hồi (responsive) của giao diện trên các thiết bị thực tế ngay trong quá trình phát triển. Dưới đây là một số giao diện của hệ thống.



Hình 5.5 Giao diện thanh toán



Hình 5.6 Giao diện quản lý loại sân của Admin

Quản lý mã nguồn bằng Git và GitHub: Toàn bộ mã nguồn giao diện HTML/CSS cùng mã xử lý Backend được quản lý phiên bản tập trung trên GitHub. Nhóm tuân thủ quy trình phân nhánh (Git Branching) tương ứng với từng tác vụ/Sprint trong quy trình Scrum...

5.6. Kiểm thử hệ thống (Test Cases)

Để đảm bảo hệ thống vận hành hoàn toàn ổn định và không phát sinh lỗi nghiêm trọng, đội ngũ phát triển đã thực hiện quy trình kiểm thử hộp đen (Black-box testing) kỹ lưỡng thông qua giao diện trực quan của Swagger UI với tổng cộng 29 kịch bản kiểm thử (Test Cases) bao phủ toàn bộ các góc cạnh của ứng dụng dưới đây:

Mã TC	Kịch bản kiểm thử chi tiết	Dữ liệu đầu vào thực nghiệm	Kết quả mong đợi từ hệ thống
TC-01	Đăng ký tài khoản hợp lệ	<i>phone, fullName, password hợp lệ, chưa tồn tại trong DB</i>	201 Created, trả về thông tin user (không bao gồm password)
TC-02	Đăng ký với số điện thoại đã tồn tại	<i>phone trùng với user đã đăng ký từ trước</i>	400 Bad Request, message: 'Số điện thoại đã tồn tại'
TC-03	Đăng ký với email đã tồn tại	<i>email trùng với user đã đăng ký từ trước</i>	400 Bad Request, message: 'Email đã tồn tại'
TC-04	Đăng nhập đúng thông tin tài khoản	<i>phone, password chính xác</i>	200 OK, trả về token JWT + thông tin user
TC-05	Đăng nhập nhập sai mật khẩu	<i>phone đúng, nhập sai password</i>	400 Bad Request, message: 'Sai mật khẩu'
TC-06	Đăng nhập với tài khoản chưa tồn tại	<i>phone chưa đăng ký trên hệ thống</i>	400 Bad Request, message: 'Tài khoản không tồn tại'
TC-07	Gọi API cần xác thực mà không có token	<i>GET /api/auth/profile, không kèm Header Authorization</i>	401 Unauthorized, message: 'Access token is missing'
TC-08	Gọi API với token không hợp lệ hoặc quá hạn	<i>Token sai định dạng hoặc đã hết hạn 7 ngày</i>	401 Unauthorized, message: 'Invalid or expired token'

Mã TC	Kịch bản kiểm thử chi tiết	Dữ liệu đầu vào thực nghiệm	Kết quả mong đợi từ hệ thống
TC-09	Khách hàng gọi API chỉ dành riêng cho Admin	<i>Token có role='customer' gọi GET /api/users</i>	403 Forbidden, message: 'Forbidden'
TC-10	Đặt sân hợp lệ, khung giờ đang trống	<i>fieldId active, ngày/giờ hợp lệ, không trùng booking khác</i>	201 Created, booking status = 'pending', có totalPrice và paymentExpiresAt
TC-11	Đặt sân với sân hiện đang bảo trì	<i>fieldId có status = 'maintenance'</i>	400 Bad Request, message: 'Sân hiện không hoạt động'
TC-12	Giờ kết thúc đặt nhỏ hơn/bằng giờ bắt đầu	<i>startTime='19:00', endTime='18:00'</i>	400 Bad Request, message: 'Giờ kết thúc phải lớn hơn giờ bắt đầu'
TC-13	Định dạng thời gian nhập bị sai	<i>startTime='7h' (không đúng định dạng HH:mm)</i>	400 Bad Request, message: 'Thời gian không hợp lệ. Định dạng phải là HH:mm'
TC-14	Đặt trùng khung giờ đã có người đặt trước	<i>Cùng fieldId, cùng ngày, khung giờ giao nhau với booking đã có</i>	400 Bad Request, message: 'Khung giờ này bị trùng với một booking khác'
TC-15	Đặt sân ở khung giờ liền kề (không giao nhau)	<i>Booking A: 18:00–20:00; đặt Booking B: 20:00–22:00</i>	201 Created (Hợp lệ vì không có khoảng thời gian giao nhau)
TC-16	Booking pending quá hạn chưa thanh toán	<i>Hiện tại $\geq$ endTime + 1 giờ (paymentExpiresAt)</i>	Hệ thống tự động chuyển status -> cancelled, paymentExpiresAt -> null, payment liên quan -> failed
TC-16b	Booking pending chưa tới hạn thanh toán	<i>Booking pending được đặt trước cho một ngày trong tương lai</i>	Đơn đặt sân vẫn giữ nguyên trạng thái pending, KHÔNG bị tự động hủy

Mã TC	Kịch bản kiểm thử chi tiết	Dữ liệu đầu vào thực nghiệm	Kết quả mong đợi từ hệ thống
TC-17	Hủy đơn đặt sân đã được hủy trước đó	Gọi lại <i>PUT</i> <i>/api/bookings/:id/cancel</i> lên đơn <i>cancelled</i>	400 Bad Request, message: 'Booking đã được hủy trước đó'
TC-18	Customer cố tình xem đơn đặt sân của người khác	<i>customerId</i> trong <i>token</i> khác với <i>customerId</i> của <i>booking</i>	403 Forbidden / Bị chặn kiểm soát tại tầng Controller
TC-18b	Tra cứu khung giờ đã đặt của một sân	<i>GET</i> <i>/api/bookings/availability?fieldId=...&bookingDate=...</i>	200 OK, trả về danh sách booking (startTime, endTime, status) chưa hủy
TC-18c	Gọi API availability nhưng thiếu tham số	Thiếu tham số <i>fieldId</i> hoặc <i>bookingDate</i>	400 Bad Request, message: 'Thiếu fieldId' hoặc thông tin tương ứng
TC-19	Thanh toán tiền mặt hợp lệ	<i>bookingId pending</i> , <i>paymentMethod='cash'</i>	201 Created, payment status = 'pending', chờ nhân viên xác nhận
TC-20	Thanh toán chuyển khoản có tải ảnh minh chứng	<i>paymentMethod='bank_transfer'</i> , kèm <i>file paymentProof</i>	201 Created, payment status = 'pending', lưu đường dẫn ảnh chuyển khoản
TC-21	Thanh toán chuyển khoản nhưng thiếu ảnh chụp	<i>paymentMethod='bank_transfer'</i> , không <i>upload file</i>	400 Bad Request, message: 'Vui lòng tải ảnh xác nhận chuyển khoản'
TC-22	Thanh toán cho đơn đặt sân đã bị hủy	<i>bookingId có status='cancelled'</i>	400 Bad Request, message: 'Booking đã bị hủy, không thể thanh toán'
TC-23	Thanh toán cho đơn đã được xác nhận từ trước	<i>bookingId có status='confirmed'</i>	400 Bad Request, message: 'Booking này không ở trạng thái chờ thanh toán'

Mã TC	Kịch bản kiểm thử chi tiết	Dữ liệu đầu vào thực nghiệm	Kết quả mong đợi từ hệ thống
TC-24	Thanh toán với phương thức không hợp lệ	<i>paymentMethod='mom o' (không nằm trong enum cho phép)</i>	400 Bad Request, message: 'Phương thức thanh toán không hợp lệ'
TC-25	Hủy đơn đặt sân đã thanh toán thành công (paid)	<i>Booking có payment.status='paid', tiến hành hủy đơn</i>	Booking -> cancelled; payment giữ nguyên paid (chờ xử lý hoàn tiền thủ công)
TC-25b	Hủy đơn đặt sân đang chờ thanh toán (pending)	<i>Booking có payment.status='pending', tiến hành hủy đơn</i>	Booking -> cancelled; payment pending liên quan tự động chuyển sang failed
TC-26	Tạo báo cáo thống kê khoảng thời gian hợp lệ	<i>fromDate, toDate hợp lệ (fromDate <= toDate)</i>	201 Created, trả về báo cáo doanh thu gồm totalRevenue, totalBookings...
TC-27	Tạo báo cáo thống kê bị thiếu ngày nhập	<i>Không truyền fromDate hoặc toDate</i>	400 Bad Request, message: 'Ngày không được để trống'
TC-28	Tạo báo cáo nhập sai định dạng ngày	<i>fromDate = '32/13/2026' (Ngày không tồn tại)</i>	400 Bad Request, message: 'Ngày không hợp lệ'
TC-29	Nhân viên (Staff) cố tình thực hiện xóa báo cáo	<i>DELETE /api/reports/:id bằng token có role='staff'</i>	403 Forbidden (Quyền xóa báo cáo chỉ dành riêng cho Admin)

5.7. Nhận xét kết quả kiểm thử

Kết quả kiểm thử thực tế cho thấy toàn bộ 29 kịch bản kiểm thử chức năng và ngoại lệ đều hoạt động cực kỳ chính xác đúng theo đặc tả nghiệp vụ đề ra. Hệ thống xử lý tốt các tình huống biên phức tạp như kiểm tra trùng khung giờ giao thoa, tự động giải phóng các ô giờ đã bị đặt chờ thanh toán pending quá hạn, đồng bộ chính xác dữ liệu giữa đơn đặt sân và hóa đơn thanh toán. Hệ thống bảo vệ tuyệt đối các endpoint API, ngăn chặn các thao tác trái phép và phản hồi mã lỗi chuẩn HTTP RESTful.

Tuy nhiên, quá trình kiểm thử cũng phát hiện một số hạn chế cần khắc phục: (1) Route quản lý loại sân /api-field-types hiện đang để Public chưa gán middleware bảo mật; (2) Việc hủy đơn đặt sân đã thanh toán paid chưa tự động cập nhật trạng thái hoàn tiền (refunded) do tính nhạy cảm giao dịch; (3) Với quy tắc giữ chỗ mới, đơn đặt sân pending ở tương lai quá xa sẽ bị giữ sân rất lâu, dễ gây hiện tượng 'treo' khung giờ ảo nếu khách hàng cố tình không thanh toán.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Dự án đã hoàn thành xuất sắc toàn bộ các mục tiêu và yêu cầu chức năng, phi chức năng đề ra ở phạm vi đề tài, cụ thể:

- Áp dụng thành công và chuẩn hóa quy trình phát triển phần mềm Agile - Scrum thông qua việc lập kế hoạch, phân chia 4 Sprints, gán Story Points và theo dõi sát sao tiến độ trên Jira.
- Xây dựng thành công RESTful API Backend bằng Node.js/Express với hơn 25 endpoints hoàn chỉnh được tài liệu hóa tự động bằng Swagger UI chuyên nghiệp tại địa chỉ /api-docs.
- Thiết kế cơ sở dữ liệu MongoDB NoSQL tối ưu với 6 collections chính, giải quyết thành công các liên kết dữ liệu logic và xử lý băm mật khẩu bảo mật bằng Bcrypt, cấp token JWT thời hạn 7 ngày.
- Giải quyết triệt để bài toán trùng lịch đặt sân bằng toán tử \$expr trong MongoDB Aggregation Pipeline đạt hiệu năng xử lý cực cao dưới 100ms.
- Cải tiến quy tắc giới hạn giữ sân (paymentExpiresAt = endTime + 1 giờ) thực tế hơn, tránh hủy sớm các booking đặt trước nhiều ngày, đồng thời xây dựng cơ chế dọn dẹp lazy expiration tự động.
- Xây dựng thành công React Frontend responsive đẹp mắt, nổi bật là giao diện chọn khung giờ dạng lưới (Time Grid) cập nhật tự động sau mỗi 30 giây trực quan và tiện dụng.
- Đóng gói hoàn toàn hệ thống bằng Docker và Docker Compose thành 3 container chạy độc lập, đồng bộ biến môi trường, giúp triển khai đồng nhất chỉ với một dòng lệnh duy nhất.

6.2. Hạn chế

Mặc dù đạt được những kết quả rất khả quan, hệ thống vẫn tồn tại một số điểm hạn chế kỹ thuật thực tế cần lưu ý:

- Route quản lý loại sân (/api/field-types) hiện chưa được bảo vệ bằng middleware authenticate/authorize, tạo ra một kẽ hở phân quyền cần được rà soát lại.

- Hệ thống chưa tích hợp cổng thanh toán trực tuyến tự động (VNPay, MoMo, ZaloPay...); việc duyệt thanh toán chuyển khoản vẫn phải dựa vào đối soát thủ công bằng mắt của nhân viên qua ảnh tải lên.
- Chưa có cơ chế gửi thông báo thời gian thực (Real-time notification) qua email, tin nhắn SMS hoặc thông báo đẩy (Push Notification) khi trạng thái đơn đặt sân hoặc thanh toán thay đổi.
- Chưa cài đặt bộ xử lý lỗi tập trung (Global Error Handler middleware) bài bản, các try-catch xử lý lỗi hiện đang viết phân tán rải rác tại các Controller khác nhau.
- Đơn đặt sân pending ở tương lai rất xa sẽ bị giữ sân rất lâu mà không có hạn chót thanh toán riêng kể từ lúc đặt, có khả năng dẫn đến việc bị lạm dụng giữ chỗ ảo làm treo khung giờ của sân.

6.3. Hướng phát triển

Để hệ thống trở nên hoàn thiện và sẵn sàng đưa vào vận hành thương mại hóa thực tế, nhóm đề xuất các hướng phát triển tiếp theo bao gồm:

- Bổ sung ngay middleware authenticate/authorize cho module loại sân và tiến hành rà soát bảo mật toàn diện tất cả các API nhạy cảm của hệ thống.
- Tích hợp cổng thanh toán trực tuyến thực tế (VNPay, MoMo, ZaloPay...) để tự động hóa hoàn toàn quy trình nhận tiền và cập nhật trạng thái đơn đặt sân ngay lập tức không qua thủ công.
- Xây dựng cơ chế gửi thông báo tự động qua Email (Sử dụng Nodemailer) hoặc qua ứng dụng chat Telegram, SMS khi đơn đặt sân được tạo, sắp hết hạn hoặc được xác nhận thành công.
- Áp dụng cơ chế kết nối thời gian thực WebSocket (Socket.io) để đồng bộ trạng thái lưới khung giờ trống của sân lập tức khi có người vừa thực hiện đặt chỗ, nâng cao trải nghiệm người dùng.
- Xây dựng giải pháp giải quyết triệt để vấn đề giữ chỗ ảo bằng cách áp dụng song song hai hạn thanh toán: Hạn thanh toán tối đa 24 giờ kể từ lúc đặt sân đối với các đơn chơi ở tương lai xa, kết hợp với hạn chơi kết thúc cộng 1 giờ như hiện tại.
- Phát triển ứng dụng di động đa nền tảng (React Native) dành riêng cho Khách hàng để hỗ trợ thao tác tra cứu và đặt sân thể thao nhanh chóng và tiện lợi hơn.

6.4. Kết luận chung

Đề tài đồ án 'Xây dựng hệ thống vận hành và quản lý sân thể thao' đã giải quyết trọn vẹn và tối ưu bài toán thực tiễn đặt ra, số hóa thành công quy trình vận hành thủ công sổ sách truyền thống của các cơ sở sân thể thao vừa và nhỏ. Việc áp dụng mô hình quản lý dự án linh hoạt Agile - Scrum giúp các thành viên phối hợp chặt chẽ, tối ưu hóa thời gian phát triển và đảm bảo chất lượng sản phẩm bàn giao sau mỗi Sprint. Đồ án là minh chứng thực chiến rõ nét cho khả năng áp dụng trọn vẹn lý thuyết môn học Công nghệ phần mềm vào việc xây dựng một sản phẩm web toàn diện, bảo mật và sẵn sàng triển khai trong thực tế.

TÀI LIỆU THAM KHẢO

- [1] Express.js. Express - Node.js web application framework. Truy cập từ: <https://expressjs.com/>
- [2] Node.js Foundation. Node.js Documentation. Truy cập từ: <https://nodejs.org/docs/latest/api/>
- [3] MongoDB, Inc. MongoDB Manual - Aggregation Pipeline & Query Operators (\$expr). Truy cập từ: <https://www.mongodb.com/docs/manual/>
- [4] Mongoose.js. Mongoose ODM v8 Documentation - Schemas, Models, Populate. Truy cập từ: <https://mongoosejs.com/docs/>
- [5] Auth0. Introduction to JSON Web Tokens (JWT). Truy cập từ: <https://jwt.io/introduction>
- [6] OpenAPI Initiative. OpenAPI Specification v3.0 (nền tảng cho Swagger). Truy cập từ: <https://swagger.io/specification/>
- [7] Meta Platforms, Inc. React Documentation (React 19). Truy cập từ: <https://react.dev/>
- [8] Vite.js. Vite - Next Generation Frontend Tooling. Truy cập từ: <https://vitejs.dev/>
- [9] Docker, Inc. Docker Documentation - Dockerfile & Docker Compose reference. Truy cập từ: <https://docs.docker.com/>